

EECT 6325 Project 1: CAN 2.0B Bus Controller bit-stream FSM

Corey Finch (CXF210015)

Adil Bhojwani (AXB210168)

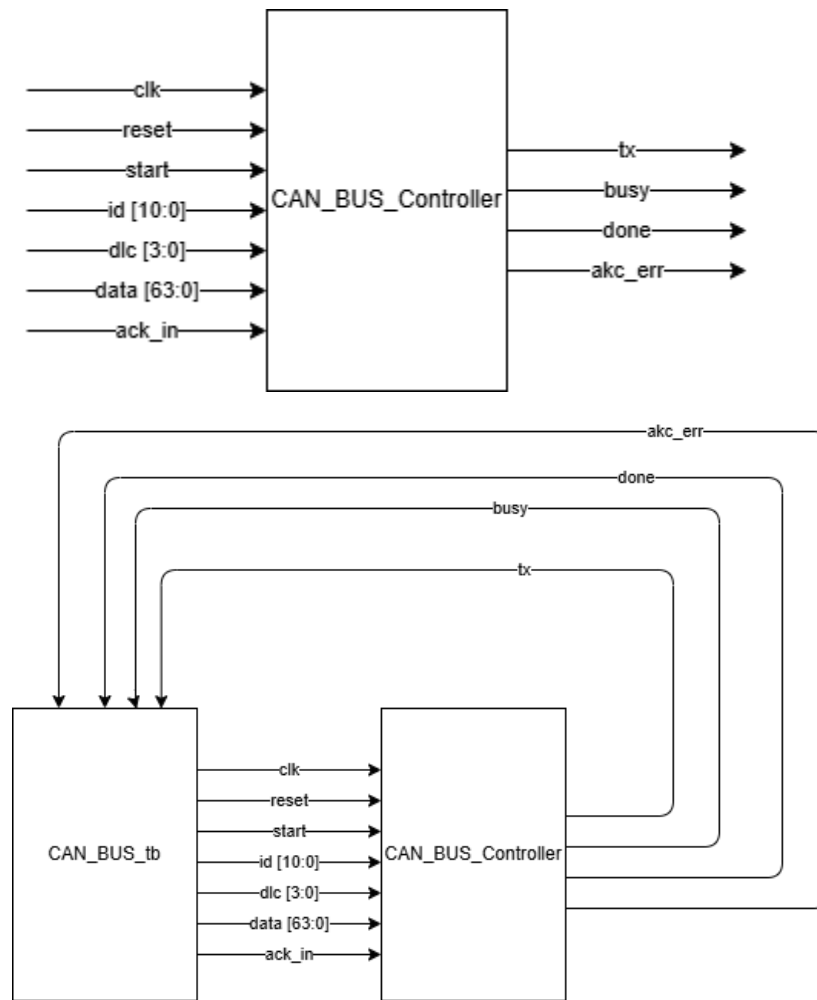
Paresh Grover (PXG210048)

Date: 9/9/2026

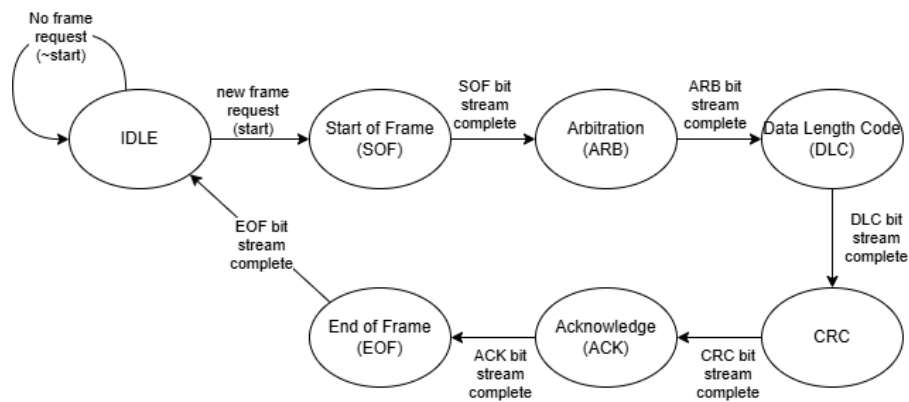
General Description:

This design implements a CAN 2.0B bus controller as a behavioral finite state machine that serializes a data frame onto a single transmit line. The module accepts an 11-bit identifier, a 4-bit data length code, a 64-bit payload, and a 15-bit CRC sequence as parallel inputs, and, upon a start pulse, transmits them bit by bit according to the CAN frame format. The FSM sequences through nine states: IDLE, Start of Frame, Arbitration, Control, Data, CRC, Acknowledge, End of Frame, and Interframe Space. A synchronous, active-high reset returns the machine to IDLE and clears all outputs to their recessive state. Each field is driven by a shared bit counter that indexes into the appropriate input register, shifting out one bit per clock edge and advancing to the next state once the field is complete; for example, the Arbitration state transmits the identifier followed by the RTR bit, while the Control state emits the IDE and reserved bits ahead of the DLC. The controller exposes a serial output, a busy flag asserted while a frame is in flight, a single-cycle done pulse marking completion, and an acknowledgment-error flag, along with a debug port for observing the current state. All parameters, including field widths and the state encoding width, are centralized in a separate parameters file, making the design straightforward to extend or scale.

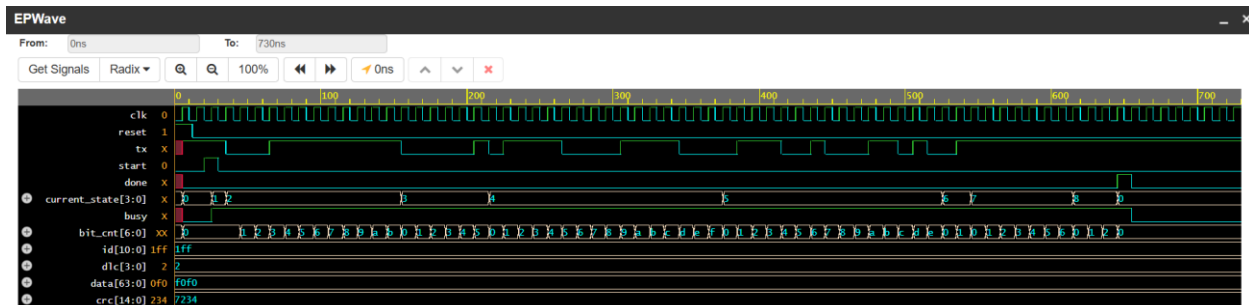
Block Diagram:



State Diagram:



Simulation Waveforms:



The signal output of the transmit buffer “tx” is emitting a full CAN frame as intended. The last bit of the field always transmits on the state’s last clock cycle + 1. The id field is seen in tx between states 2 (Arbitration state) and 3 (Control state) with the last bit (field RTR) on the first clock cycle of state 3. The rest of the frame fields follow the same trend, indicating a correct transmitted output.

Simulation Code:

```

/*
    CAN 2.0B bus controller - bit-stream FSM

    Missing features:
    - bit stuffing
    - CRC generation
    - error handling
*/

`include "Params.sv"

module CAN_BUS (
    /* Inputs */
    input clk,
    input reset,
    input start,                // pulse to request frame transmission
    input [ID_WIDTH-1:0] id,    // 11-bit Unique Identifier
    input [DLC_WIDTH-1:0] dlc,  // Data Length Code, number of data bytes
                                // (0-8)
    input [DATA_WIDTH-1:0] data, // Payload data, MSB of byte 0 sent first
    input ack_in,               // bus level sampled during the ACK slot

    /* Outputs */
    output reg tx,              // serial bus output (0 = dominant, 1 =
                                // recessive)
    output reg busy,           // high while a frame is in flight
    output reg done,           // one-cycle pulse at end of IFS

```

```

    output reg ack_err          // set if no receiver acknowledged
);

/* FSM States */
localparam IDLE      = 4'd0; // IDLE state, waiting for a frame to transmit
localparam SOF       = 4'd1; // Start of Frame, transmit a dominant bit
localparam ARB       = 4'd2; // Arbitration, resolve bus contention
localparam CTRL      = 4'd3; // Control, transmit control bits (IDE, r0,
DLC)
localparam DATA      = 4'd4; // Data, transmit the payload data
localparam CRC        = 4'd5; // Cyclic Redundancy Check, verify frame
integrity
localparam ACK        = 4'd6; // Acknowledge, check for receiver
acknowledgment
localparam EOF        = 4'd7; // End of Frame, transmit the end of frame
sequence
localparam IFS        = 4'd8; // Interframe Space, wait before next frame

reg [NUM_STATES_WIDTH-1:0] current_state;
reg [BIT_CNT_WIDTH-1:0] bit_cnt; // Count bits transmitted in the current
field
reg [CRC_WIDTH-1:0] crc = 15'h7234; // Arbitrary CRC (Not currently
calculated)

task reset_all_outputs;
    begin// Reset all outputs and internal signals
        tx <= 1'b1;          // Recessive state
        busy <= 1'b0;        // Not busy
        done <= 1'b0;        // Not done
        ack_err <= 1'b0;     // No acknowledgment error
        bit_cnt <= {BIT_CNT_WIDTH{1'b0}}; // Reset bit counter
    end
endtask

always @(posedge clk) begin
    if (reset) begin
        // Reset all outputs and internal signals
        reset_all_outputs();
        current_state <= IDLE; // Go to IDLE state
    end
    else begin
        done <= 1'b0;
        case (current_state)
            IDLE: begin

```

```

        // Stay in IDLE state waiting for a frame to transmit
        tx <= 1'b1;
        busy <= 1'b0;
        // Move to the Start of Frame state once new frame is
requested
        if (start) begin
            bit_cnt <= {BIT_CNT_WIDTH{1'b0}};
            busy <= 1'b1; // Set busy when a new frame is requested
            ack_err <= 1'b0; // Clear the previous frame's result
            current_state <= SOF; // Move to Start of Frame state
        end
    end
    SOF: begin
        // Transmit a dominant bit (0) for the Start of Frame
        tx <= 1'b0;
        bit_cnt <= {BIT_CNT_WIDTH{1'b0}}; // Reset bit counter
        current_state <= ARB; // Move to the Arbitration state
    end
    ARB: begin
        // Transmit the arbitration field (ID 11 + RTR 1).
        // RTR is dominant (0) for data frames.
        tx <= (bit_cnt < ID_WIDTH) ? id[ID_WIDTH-1-bit_cnt] : 1'b0;
        // Leave on the same clock that drives the last bit, so no
        // bit time is spent on the state change itself.
        if (bit_cnt == ID_WIDTH) begin
            bit_cnt <= {BIT_CNT_WIDTH{1'b0}};
            current_state <= CTRL; // Move to the Control state
        end
        else bit_cnt <= bit_cnt + 1;
    end
    CTRL: begin
        // Transmit the control bits (IDE, r0, DLC)
        tx <= (bit_cnt < 2) ? 1'b0 // IDE = 0, r0 = 0
            : dlc[DLC_WIDTH-1+2-bit_cnt]; // DLC
bits
        if (bit_cnt == DLC_WIDTH+1) begin
            bit_cnt <= {BIT_CNT_WIDTH{1'b0}};
            current_state <= DATA; // Move to the Data state
        end
        else bit_cnt <= bit_cnt + 1;
    end
    DATA: begin
        // Transmit the payload data, MSB of byte 0 first.
        tx <= data[(dlc*8)-1 - bit_cnt];
        if (bit_cnt == (dlc*8)-1) begin

```

```

        bit_cnt <= {BIT_CNT_WIDTH{1'b0}};
        current_state <= CRC; // Move to the CRC state
    end
    else bit_cnt <= bit_cnt + 1;
end
CRC: begin
    // Transmit the CRC sequence
    tx <= crc[CRC_WIDTH-1 - bit_cnt];
    if (bit_cnt == CRC_WIDTH-1) begin
        bit_cnt <= {BIT_CNT_WIDTH{1'b0}};
        current_state <= ACK; // Move to the Acknowledge state
    end
    else bit_cnt <= bit_cnt + 1;
end
ACK: begin
    // The transmitter leaves both the ACK slot and its
    // delimiter recessive; a receiver pulls the slot dominant.
    tx <= 1'b1;
    if (bit_cnt == 0) bit_cnt <= bit_cnt + 1;
    else begin
        // tx is registered, so the slot assigned last clock is
        // only on the bus now - this is when ack_in is valid.
        // Still recessive means nobody acknowledged.
        ack_err <= ack_in;
        bit_cnt <= {BIT_CNT_WIDTH{1'b0}};
        current_state <= EOF; // Move to the End of Frame state
    end
end
EOF: begin
    // Transmit the end of frame sequence (7 recessive bits)
    tx <= 1'b1;
    if (bit_cnt == EOF_WIDTH-1) begin
        bit_cnt <= {BIT_CNT_WIDTH{1'b0}};
        current_state <= IFS; // Move to the Interframe Space
state

    end
    else bit_cnt <= bit_cnt + 1;
end
IFS: begin
    // Hold the bus recessive for the interframe space before
    // allowing a new frame, then go back to IDLE.
    tx <= 1'b1;
    if (bit_cnt == IFS_WIDTH-1) begin
        bit_cnt <= {BIT_CNT_WIDTH{1'b0}};
        current_state <= IDLE;

```

```
        done <= 1'b1; // Signal that the frame transmission is
done
        end
        else bit_cnt <= bit_cnt + 1;
    end
    default: begin
        // Default case, go back to IDLE
        current_state <= IDLE;
    end
endcase
end
end
endmodule
```